

SpringBoot

CRUD Operations :-

- CRUD means Create, Read, Update, Delete.
- These are the basic operations for working with stored data.
- CRUD actions map to HTTP methods called action verbs.
- HTTP methods act like CRUD operations in web development.
- They help developers handle web apps and databases easily.

1. POST: Creates a new resource
2. GET: Reads/Retrieve a resource
3. PUT: Updates an existing resource
4. DELETE: Deletes a resource



1. CREATE Operation: Performs the INSERT statement to create a new record.
2. READ Operation: Reads table records based on the input parameter.
3. UPDATE Operation: Executes an update statement on the table. It is based on the input parameter.
4. DELETE Operation: Deletes a specified row in the table. It is also based on the input parameter.

Spring Boot CrudRepository :-

- CrudRepository is an interface in Spring Boot that provides built-in methods for basic CRUD operations.
- It belongs to the package `org.springframework.data.repository` and extends the Repository interface.
- To use it, you create your own interface and extend CrudRepository.

Example -

```
public interface DepartmentRepository extends CrudRepository<Department, Long> { }
```

Where:

- **Department** → The entity/model class managed by the repository
- **Long** → The data type of the entity's **@Id** field

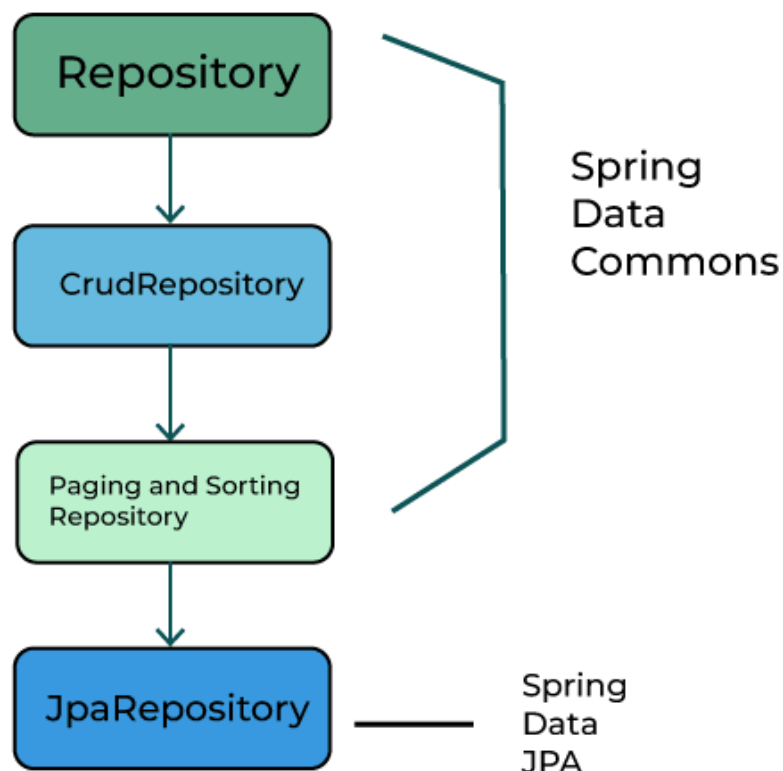
Spring Boot JpaRepository :-

- JPA stands for Java Persistence API.
- JpaRepository is an extended version of the basic repository.
- It is defined in `org.springframework.data.jpa.repository`.
- It provides JPA-specific methods like flushing the persistence context and batch deletion.
- JpaRepository supports CRUD operations, pagination, and sorting.
- With JpaRepository, we don't need to write DDL/DML queries manually; XML/annotations can be used instead.

Syntax –

```
public interface JpaRepository<T, ID>  
extends PagingAndSortingRepository<T, ID>, QueryByExampleExecutor<T>
```

Spring Data Repository Interface :-



Difference between CRUD Repository and Jpa Repository :-

Feature	CrudRepository	JpaRepository
Purpose	Basic CRUD operations	CRUD + JPA-specific operations
Package	org.springframework.data.repository	org.springframework.data.jpa.repository
Features	Create, Read, Update, Delete	CRUD + Pagination + Sorting + Batch operations
Advanced Methods	Not available	flush(), saveAllInBatch(), deleteAllInBatch()
Use Case	Simple applications	Complex applications needing JPA features
Query Support	Limited	Supports JPA queries, annotations, and custom methods
Performance	Lightweight	More powerful but slightly heavier
Extends	Repository	PagingAndSortingRepository + JPA executor

Example of Spring Boot - CRUD Operations :-

Step 1: Create a Spring Boot project.

Step 2: Add the following dependency

1. Spring Web
2. H2 Database
3. Lombok
4. Spring Data JPA

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.0.0</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.example</groupId>
  <artifactId>Spring_Boot_Sample_Project1</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>Spring_Boot_Sample_Project1</name>
  <description>Demo project for Spring Boot</description>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies> Add Spring Boot Starters...
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

```

<!-- https://mvnrepository.com/artifact/org.apache.tomcat.embed/tomcat-embed-jasper -->
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
  <scope>provided</scope>
</dependency>

<!-- jsp pages-->
<!-- https://mvnrepository.com/artifact/jakarta.servlet.jsp.jstl/jakarta.servlet.jsp.jstl-api -->
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
</dependency>

<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
</dependency>

<!--Database-->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>

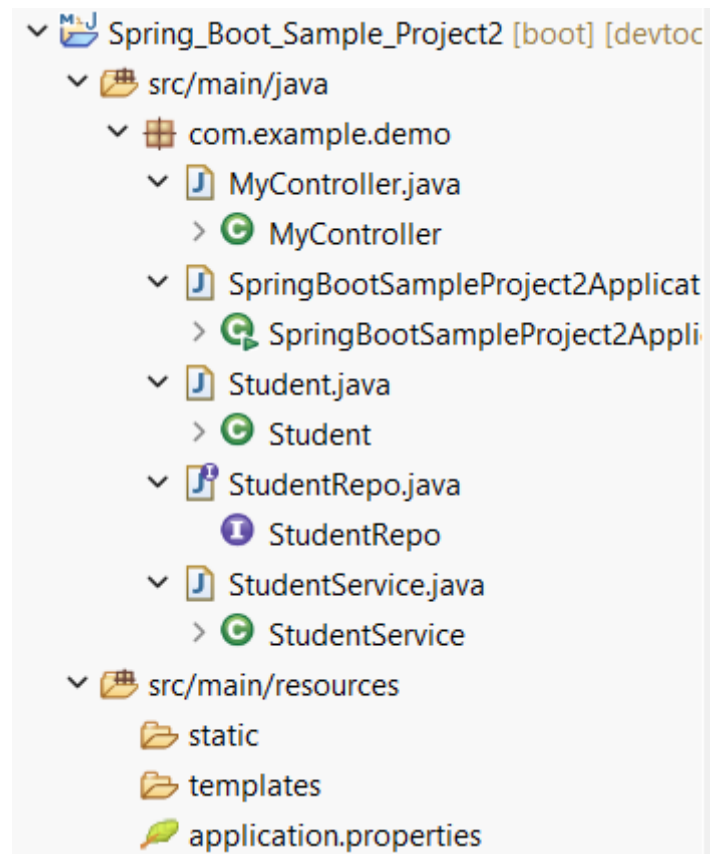
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

Step 3: Create package and later create some classes and interfaces inside these packages as seen in the below image:

- Controller
- Application
- repository
- service



Step 4: Inside the entity package

Create a simple [POJO class](#) inside the Student.java file

```
package com.example.demo;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column
    private String fullname;

    @Column(unique=true)
    private String email;

    @Column
    private String password;

    @Column
    private String address;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getFullname() {
        return fullname;
    }

    public void setFullname(String fullname) {
        this.fullname = fullname;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    @Override
    public String toString() {
        return "Student [id=" + id + ", fullname=" + fullname + ", email=" + email + ", password=" + password
            + ", address=" + address + "];"
    }
}
```

Step 5: Inside the repository package

Create a simple interface and name the interface as StudentRepo. This interface is going to extend the CrudRepository as we have discussed above. Below is the code for the StudentRepo.java file

```
package com.example.demo;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepo extends JpaRepository<Student, Long> {

}
```

Step 6: Inside the service package

Inside the package create one interface named as StudentService and one class named as StudentService.java

```
package com.example.demo;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class StudentService {

    @Autowired
    StudentRepo sr;

    public void registration(Student s)
    {
        sr.save(s);    //insert
    }

    public List<Student> getallstudents(){
        return sr.findAll();    //select * from student
    }

    public void delete(Long id) {
        //sr.deleteById(id);

        if(sr.existsById(id)) {
            sr.deleteById(id);
        }else {
            throw new RuntimeException("Id not found");
        }
    }

    public Optional<Student> getStudentById(Long id) {
        return sr.findById(id);
    }

    public void updateStudent(Student s) {
        sr.save(s);
    }

}
```

Step 7: Inside the controller package

Inside the package create one class named as MyController

```
package com.example.demo;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class MyController {

    @Autowired
    StudentService ss;

    @GetMapping("/")
    public String home()
    {
        return "home"; //home.jsp
    }

    @GetMapping("/register")
    public String register()
    {
        return "register"; //register.jsp
    }

    @GetMapping("/viewdata")
    public String viewdata(Model m) {
        List<Student> list = ss.getAllStudents();
        m.addAttribute("list", list);
        return "viewdata"; //viewdata.jsp
    }

    @GetMapping("/login")
    public String login()
    {
        return "login"; //login.jsp
    }

    @PostMapping("/saveform")
    public String saveform(@ModelAttribute Student s) {
        ss.registration(s);
        return "redirect:viewdata";
    }

    @GetMapping("/delete/{id}")
    public String delete(@PathVariable Long id) {
        try {
            ss.delete(id);
        } catch (Exception e) {
            System.out.println(e);
        }
        return "redirect:viewdata";
    }

    @GetMapping("/profileedit/{id}")
    public String profileedit(@PathVariable Long id, Model m) {
        Optional<Student> s= ss.getStudentById(id);
        m.addAttribute("student",s.get());
        return "edit"; //edit.jsp
    }

    @PostMapping("/update/{id}")
    public String updateStudent(@PathVariable Long id, @ModelAttribute Student s) {
        s.setId(id);
        ss.updateStudent(s);
        return "redirect:viewdata";
    }
}
```

Step 8: Below is the code for the application.properties file

```
spring.application.name=Spring_Boot_Sample_Project2

spring.mvc.view.prefix=/WEB-INF/pages/
spring.mvc.view.suffix=.jsp

spring.web.resources.static-locations=classpath:/static/
spring.mvc.static-path-pattern=/resources/**

spring.datasource.url=jdbc:postgresql://localhost:5433/springbootdb
spring.datasource.username=postgres
spring.datasource.password=4518

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Step 9: To add jsp pages.

home.jsp -

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGLyvwx9Hj09JcYn3nv7wiPVLz7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous"></script>
</head>
<body>

    <%@ include file="header.jsp" %>

    <h1>Home Page</h1>

</body>
</html>
```

register.jsp –

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGLyvwx9Hj09JcYn3nv7wiPVLz7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous"></script>
</head>
<body>

    <%@ include file="header.jsp" %>

    <div class="row">
        <div class="col-sm-4"></div>
        <div class="col-sm-4">
            <h1>Registration Form</h1><br>

            <form action="saveform" method="post">
                <input type="text" name="fullname" placeholder="Enter your fullname" class="form-control mb-2">
                <input type="email" name="email" placeholder="Enter your email" class="form-control mb-2">
                <input type="password" name="password" placeholder="Enter your password" class="form-control mb-2">
                <textarea name="address" placeholder="Enter your address" class="form-control mb-2"></textarea>
                <button type="submit" class="btn btn-success">Submit</button>
            </form>
        </div>
    </div>
</body>
</html>
```

viewdata.jsp –

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
</html>
</head>
<meta charset="UTF-8">
<title>Insert title here</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNwLzeN8tE5YBuJZqJLB" crossorigin="anonymous">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGLyvwvx9Hj09JcYn3nv7wiPVLz7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous"></script>
</head>
</body>

<%@ include file="header.jsp" %>

<h1>Students Data</h1>

<table class="table">
<tr>
<th>Id</th>
<th>Fullname</th>
<th>Email</th>
<th>Password</th>
<th>Address</th>
<th></th>
</tr>

<c:forEach var="s" items="${list}" varStatus="status">
<tr>
<td>${status.index + 1}</td>
<td>${s.fullname}</td>
<td>${s.email}</td>
<td>${s.password}</td>
<td>${s.address}</td>

<td>
<a href="delete/${s.id}" class="btn btn-danger" onClick="return confirm('Are you sure ?')">Delete</a>
<a href="profileedit/${s.id}" class="btn btn-success">Profile Edit</a>
</td>
</tr>
</c:forEach>
</table>
</body>
</html>
```

edit.jsp –

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-sRI4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNwLzeN8tE5YBujZqJLB" crossorigin="anonymous">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGLyvvwx9Hj09JcYn3nv7wiPVLz7YYwJrWVCXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous"></script>
</head>
<body>

    <%@ include file="header.jsp" %>

    <div class="row">
        <div class="col-sm-4"></div>
        <div class="col-sm-4">
            <h1>Profile Edit Form</h1><br>

            <form action="/update/${student.id}" method="post">
                <input type="text" name="fullname" placeholder="Enter your fullname" class="form-control mb-2" value="${student.fullname}">
                <input type="email" name="email" placeholder="Enter your email" class="form-control mb-2" value="${student.email}">
                <input type="password" name="password" placeholder="Enter your password" class="form-control mb-2" value="${student.password}">
                <textarea name="address" placeholder="Enter your address" class="form-control mb-2">${student.address}</textarea>
                <button type="submit" class="btn btn-success">Submit</button>
            </form>
        </div>
    </div>
</body>
</html>
```

login.jsp –

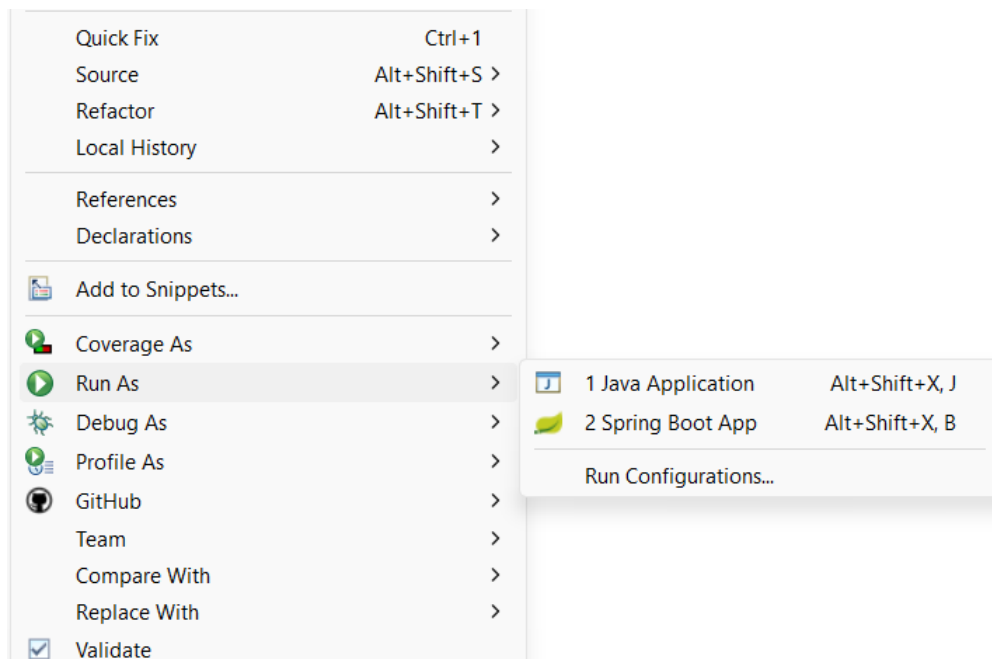
```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-sRI4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNwLzeN8tE5YBujZqJLB" crossorigin="anonymous">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGLyvvwx9Hj09JcYn3nv7wiPVLz7YYwJrWVCXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous"></script>
</head>
<body>

    <%@ include file="header.jsp" %>

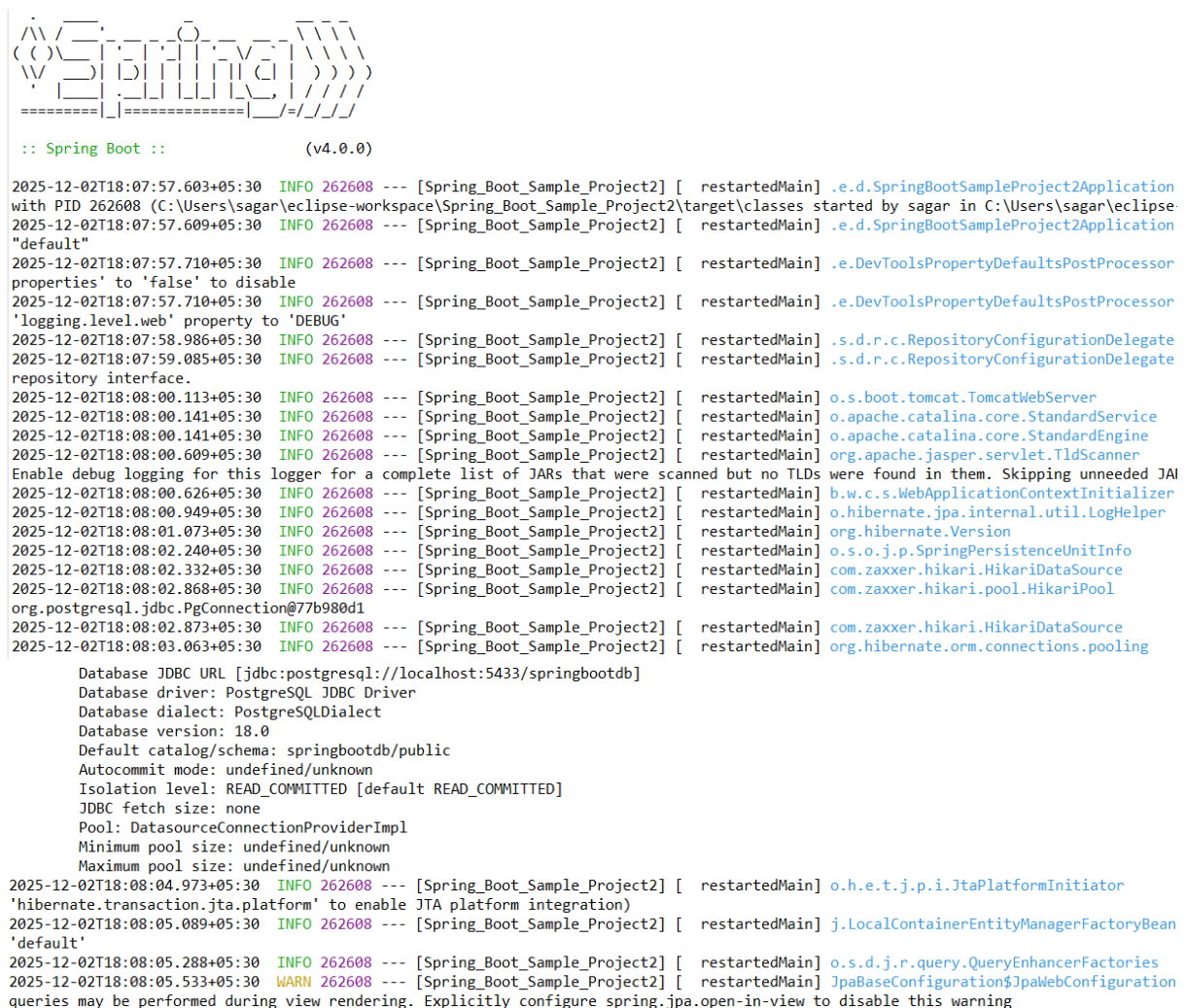
    <div class="row">
        <div class="col-sm-4"></div>
        <div class="col-sm-4">
            <h1>Login Form</h1><br>

            <form action="saveform" method="post">
                <input type="email" name="email" placeholder="Enter your email" class="form-control mb-2">
                <input type="password" name="password" placeholder="Enter your password" class="form-control mb-2">
                <button type="submit" class="btn btn-success">Submit</button>
            </form>
        </div>
    </div>
</body>
</html>
```

Run the Application :-



Console Based –



```

2025-12-02T18:08:06.265+05:30 INFO 262608 --- [Spring_Boot_Sample_Project2] [ restartedMain] o.s.boot.tomcat.TomcatWebServer
2025-12-02T18:08:06.278+05:30 INFO 262608 --- [Spring_Boot_Sample_Project2] [ restartedMain] .e.d.SpringBootSampleProject2Application
(process running for 11.019)
2025-12-02T18:08:18.139+05:30 INFO 262608 --- [Spring_Boot_Sample_Project2] [nio-8080-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/]
2025-12-02T18:08:18.140+05:30 INFO 262608 --- [Spring_Boot_Sample_Project2] [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet
2025-12-02T18:08:18.143+05:30 INFO 262608 --- [Spring_Boot_Sample_Project2] [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet
2025-12-02T18:08:20.419+05:30 WARN 262608 --- [Spring_Boot_Sample_Project2] [nio-8080-exec-1] o.s.web.servlet.PageNotFound
2025-12-02T18:08:20.435+05:30 WARN 262608 --- [Spring_Boot_Sample_Project2] [nio-8080-exec-1] o.s.web.servlet.PageNotFound

```

```

Hibernate:
    select
        s1_0.id,
        s1_0.address,
        s1_0.email,
        s1_0.fullname,
        s1_0.password
    from
        student s1_0

```

```

Hibernate:
    select
        s1_0.id,
        s1_0.address,
        s1_0.email,
        s1_0.fullname,
        s1_0.password
    from
        student s1_0

```

```

Hibernate:
    insert
    into
        student
        (address, email, fullname, password)
    values
        (?, ?, ?, ?)

```

```

Hibernate:
    select
        s1_0.id,
        s1_0.address,
        s1_0.email,
        s1_0.fullname,
        s1_0.password
    from
        student s1_0

```

```

Hibernate:
    select
        count(*)
    from
        student s1_0
    where
        s1_0.id=?

```

```

Hibernate:
    select
        s1_0.id,
        s1_0.address,
        s1_0.email,
        s1_0.fullname,
        s1_0.password
    from
        student s1_0

```

```

    where
        s1_0.id=?

```

```

Hibernate:
    delete
    from
        student
    where
        id=?

```

```

Hibernate:
    select
        s1_0.id,
        s1_0.address,
        s1_0.email,
        s1_0.fullname,
        s1_0.password
    from
        student s1_0

```

```

Hibernate:
    select
        s1_0.id,
        s1_0.address,
        s1_0.email,
        s1_0.fullname,
        s1_0.password
    from
        student s1_0
    where
        s1_0.id=?

```

```

Hibernate:
    update
        student
    set
        address=?,
        email=?,
        fullname=?,
        password=?
    where
        id=?

```

```
Hibernate:
  select
    s1_0.id,
    s1_0.address,
    s1_0.email,
    s1_0.fullname,
    s1_0.password
  from
    student s1_0
```

Web Based –

Register Page – Insert Data

localhost:8080/register

Navbar Home Register Students View Data Login

Registration Form

Sagar Kate

sagarkate374@gmail.com

Satara

Submit

Viewdata Page – Retrive/Fetch Data

localhost:8080/viewdata

Navbar Home Register Students View Data Login

Students Data

Id	Fullname	Email	Password	Address		
1	Sagar Kate	sagarkate374@gmail.com	8798	Satara	Delete	Profile Edit
2	Ganesh Kate	ganeshkate564@gmail.com	5456	Koregaon	Delete	Profile Edit
3	Rahul Kate	rahulkate7979@gmail.com	8785	Pune	Delete	Profile Edit

Delete Data –

localhost:8080/viewdata

Navbar Home Register Students View Data Login

Students Data

localhost:8080 says
Are you sure ?
OK Cancel

Id	Fullname	Email	Password	Address		
1	Sagar Kate	sagarkate374@gmail.com	8798	Satara	Delete	Profile Edit
2	Ganesh Kate	ganeshkate564@gmail.com	5456	Koregaon	Delete	Profile Edit
3	Rahul Kate	rahulkate7979@gmail.com	8785	Pune	Delete	Profile Edit

After Delete Data –

localhost:8080/viewdata

Navbar Home Register Students View Data Login

Students Data

Id	Fullname	Email	Password	Address		
1	Sagar Kate	sagarkate374@gmail.com	8798	Satara	Delete	Profile Edit
2	Rahul Kate	rahulkate7979@gmail.com	8785	Pune	Delete	Profile Edit

Profile Edit Page –

[←](#) [→](#) [🔄](#) [🔒](#) localhost:8080/profileedit/13

Navbar [Home](#) [Register](#) [Students View Data](#) [Login](#)

Profile Edit Form

Updata Data –

[←](#) [→](#) [🔄](#) [🔒](#) localhost:8080/viewdata

Navbar [Home](#) [Register](#) [Students View Data](#) [Login](#)

Students Data

Id	Fullname	Email	Password	Address		
1	Rahul Kate	rahulkate7979@gmail.com	8785	Pune	Delete	Profile Edit
2	Sagar Kate	sagarkate375@gmail.com	8794	Satara	Delete	Profile Edit

DataBase – Postgresql

	id [PK] bigint 	address character varying (255) 	email character varying (255) 	fullname character varying (255) 	password character varying (255) 
1	13	Satara	sagarkate375@gmail.com	Sagar Kate	8794
2	19	Pune	rahulkate7979@gmail.c...	Rahul Kate	8785